

Asynchronous Peer Learning for Long-Horizon Agents

Project idea summary — October 6, 2026

1 Problem and research question

Long-horizon agents reason, use tools, and respond to environment feedback. Outcome-only reinforcement learning assigns a coarse signal to many actions. Peers may acquire complementary behaviors, but their guidance can be misleading and costs time to score and communicate.

This project asks whether **asynchronous, reward-calibrated peer learning** improves population mean success and learning efficiency after accounting for its cost. Agents solve tasks and update independently, exchanging frozen policy snapshots to score their own local trajectories.

2 Core method

All K agents start identically and keep separate environments, LoRA parameters, and optimizers. Each batch contains 16 tasks and eight complete trajectories per task, generated by one frozen behavior policy. Task-specific evaluators give terminal rewards of +1 for success and -1 for failure.

Local learning. Without a process reward model (PRM), Group Relative Policy Optimization (GRPO) normalizes each task’s eight terminal rewards using their sample standard deviation. The trajectory advantage is shared by its action tokens; constant-reward groups remain with zero GRPO advantage. With PRM, compose terminal and process rewards before normalization by task and interaction-step index.

Asynchronous exchange. Clear the inbox at batch start. At reward/advantage completion, select the latest snapshot received in that batch, breaking latest-arrival ties uniformly. Lock the teacher through scoring and optimization; skip peer on-policy distillation (OPD) if the inbox is empty. Broadcast after every completed update, without a global round barrier.

Reward calibration. For trajectory i with nonempty calibration tokens $\mathcal{F}_i$, compare teacher and old-policy support on the same actual prefixes:

$$J_i = \frac{1}{|\mathcal{F}_i|} \sum_{t \in \mathcal{F}_i} [\log \pi_T(y_{it} \mid h_{it}) - \log \pi_{\text{old}}(y_{it} \mid h_{it})]. \quad (1)$$

Keep peer guidance when $R_i J_i > 0$ and reverse it when $R_i J_i < 0$; use $\lambda_i = +1$ at $J_i = 0$. Here $\mathcal{F}_i$ contains final-answer tokens for mathematics and all effective generated reasoning/action tokens for SWE, Terminal, and GUI. An empty mathematics answer disables peer OPD. At each old-policy top-four candidate v ,

$$A_{itv}^{\text{peer}} = \text{stopgrad}[\lambda_i w_{itv} \text{clip}(\log \pi_T(v \mid h_{it}) - \log \pi_{\text{old}}(v \mid h_{it}), -C, C)], \quad (2)$$

where w_{itv} normalizes old-policy probabilities over the four candidates. Clip GRPO and OPD separately, then combine as $\mathcal{L} = \alpha \mathcal{L}_{\text{GRPO}} + \beta \mathcal{L}_{\text{peer}} + 0.01 \mathcal{L}_{\text{ref}}$. Losses average action tokens within each complete trajectory, then average trajectories equally. Only local trainable parameters receive gradients; a snapshot never overwrites the receiver’s policy.

3 Four experiments

The four domains test peer learning across reasoning, code repair, shell interaction, and visual control. These choices follow current project records; pilot budgets remain provisional.

1. Mathematics with Python. Agents reason and execute Python in isolated persistent sessions. Use 2,000 training and 500 validation problems from MATH train; test on MATH-500’s 500 problems. A fixed strong model judges answers against references and supplies optional process scores. The pilot uses eight Qwen3-1.7B agents. Measure answer accuracy averaged over agents; calibrate teachers on final-answer tokens.

2. SWE repository repair. Agents inspect code, edit files, run tests, and submit patches. The planned SWE-smith-py split is 1,600/200/200 train/validation/test problems, targeting new problems in the same repositories. Official fail-to-pass/pass-to-pass evaluation supplies terminal rewards and the resolved-task rate. The framework uses four agents; the learner, split grouping, and run configuration remain pending. Calibrate on the complete generated reasoning/action trajectory.

3. Terminal tasks. Agents operate Harbor Docker/Compose shell sandboxes using SETA-Env and Terminal-Bench interfaces. Dataset revisions and task-family splits remain pending. Native verifiers determine success; external models supply optional process rewards. The pilot uses eight Qwen3-1.7B agents. Measure success and success within specified time budgets; calibrate on all effective generated reasoning/action tokens.

4. GUI tasks. Eight Qwen3-VL-4B-Thinking agents use screenshots, mouse, and keyboard, training language-only LoRA. Use 3,516 compatible CUA-Gym Office tasks for training, 200 held-out-family tasks for validation, and the 117-task OSWorld-Verified Office subset for external testing. Native evaluators supply terminal rewards; a frozen visual model supplies optional process scores. Measure mean native success, label the Office subset explicitly, and calibrate on generated reasoning/actions.

3.1 Common comparisons and measurements

The planned factorial comparison separates peer OPD from PRM:

Condition	Peer OPD	PRM	Local training signal
A	Off	Off	Outcome GRPO + reference KL
B	On	Off	A + calibrated peer OPD
C	Off	On	Outcome/process GRPO + reference KL
D	On	On	C + calibrated peer OPD

Compare B–A and D–C for peer effects, and C–A and D–B for PRM. Match initialization, splits, sampling, batches, and evaluation within each domain. Record mean success, time to target success, samples/tokens, GPU/scoring/communication cost, and individual/mean update and OPD counts. Distinguish matched updates from matched wall-clock or compute budgets.

Mathematics, Terminal, and GUI pilots use ten updates per agent. Formal budgets, repeated seeds, and pending settings must be fixed before drawing conclusions. Existing records cover implementation and local validation; full-model training results are not yet available.

4 Planned theoretical work

The project will study the asynchronous learning system through update and OPD counts, accounting for random computation times, snapshot transmission delays, and the existing inbox and teacher-selection rules. Within a specified time horizon, we will analyze how many parameter updates and OPD processes a single agent completes, the population average of these counts, and the proportion of updates that include OPD. We will develop probability guarantees and investigate how asynchronous interaction, communication delays, teacher availability, and clearing the inbox affect these quantities and their variability. We will also study whether update rates and OPD frequencies converge as running time grows, identify the assumptions needed for such limits, and examine how local learning and peer-scoring costs influence long-run behavior. As the number of agents grows, we will analyze changes in counts, rates, and fluctuations, including possible synchronization and dependencies between agents, while distinguishing fixed resources per agent from a fixed total compute budget. This work forms the planned theoretical agenda; specific formulas, bounds, and asymptotic conclusions will be developed separately with explicit assumptions.